

Ikusiko dugu Androiden plataformak hainbat erraztasun ematen dizkigula datuak etengabe biltegitatzeko (hau da, datuak ez direla ezabatzen aplikazioa itzaltzen denean). Behar motaren arabera, metodo hauetakoren bat erabiliko dugu:

- ✓ SharedPreferences klasearen bidez.
- ✓ Testu-fitxategien bidez.
- ✓ SD tarjeta bidez
- ✓ Datu-base batean

Ez da arraroa izango aplikazio batek metodo horietako bat baino gehiago erabiltzea datuak biltegitatzeko.

Datu kopuru mugatu bat biltegitatu behar dugunean, egokia da **SharedPreferences** klasea erabiltzea. Adibidez, aplikazioaren konfigurazioak: pantaila-koloreak, joko baten egungo maila, datuen sarrera-kontrolen hasierako datuak, etab.

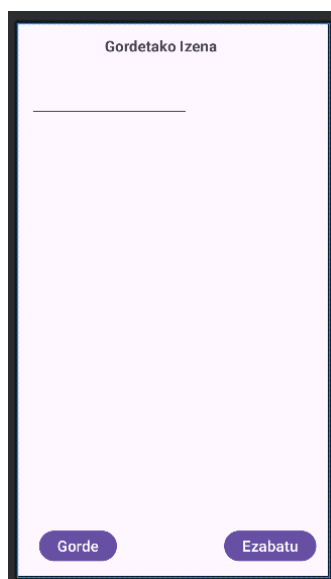
Shared-ak informazio kantitate txikiak biltzera bideratuta daude, iltze-balio bidez. Konprometitu gabeko informazioa izan behar du, Androidek XML fitxategi bat sortuko baitu informazio hori guztia zifratu gabe gordetzeko. Fitxategi horiek partekatuak edo pribatuak izan daitezke.

PRAKTIKA 07-00

SharedPreferences

Proiektu honetan, oso aplikazio erraza sortuko dugu, eta, bertan, izenik gordeta ez badago, jartzeko eskatuko digu; bat badugu ordea, pantailan agertuko zaigu, ezabatzeko botoi baten ondoan.

Lehenik eta behin, proiektu berri bat sortuko dugu, eta layout-aren maketazio sinple bat egingo dugu.



The screenshot shows a mobile application interface with a light pink background. At the top, there is a text label "Gordetako Izena". Below it is a text input field with a horizontal line. At the bottom, there are two rounded rectangular buttons: "Gorde" on the left and "Ezabatu" on the right, both with a purple gradient.

Ondoren, klase bat sortuko dugu, shared-arekin lan egiteko behar den guztia definitzeko. **Prefs** deituko diogu. Klase honek testuinguru bat izango du, kasu honetan aplikazioarena, behin bakarrik eskatu ahal izateko aplikazioa hastean eta pref objektu bat izateko.

```
import android.content.Context
import android.content.SharedPreferences

class Prefs (context: Context) {
    val PREFS_NAME = "com.example.praktika07_datuakgorde.sharedpreferences"
    val SHARED_NAME = "shared_name"
    val prefs: SharedPreferences = context.getSharedPreferences(PREFS_NAME, 0)
    var name: String
        get() = prefs.getString(SHARED_NAME, "").toString()
        set(value) = prefs.edit().putString(SHARED_NAME, value).apply()
}
```

Lehenik eta behin, bi konstante definitu ditugu: PREFS_NAME eta SHARED_NAME. Lehenengoa aurrerago sortuko dugun pref objektuaren gakoa izango da, eta bigarrena biltegitratuko dugun izenaren gakoa. Gogoratu shared preference-ak iltze-balioarekin biltegitratzen direla, eta horrek esan nahi du «name» balioa eskatzeko SHARED_NAME gakoaren bidez eskatu behar dugula.

Ikusi, halaber, name aldagai bat definitu dugula, eta hor gordeko dugula izena, lehen esan dudana bezala, baina get eta set metodoa gainidatzi dugula; beraz, name balioa eskatzen dugunean, horrek objektu prefera joko du, eta SHARED_NAME kodeari dagokion balio hori eskatuko du. Gauza bera set-arekin, eta prefs.edit() .putString (SHARED_NAME, value) .apply () bidez, esaten dizugun balioa gordeko du. Jakina, beste aldagai mota bat balitz, adibidez Int bat, putString-a () putInt () batez aldatuko genuke eta horrela aldagai mota bakoitzarekin.

Orain gure MainActivity-ra joango gara, falta den logika pixka bat garatzera.

Asmoa bi metodo sortzea da, batek gonbidatuentzako bista bat erakutsiko du eta besteak profilaren ikuspegia. Horien arteko aldea izango da gonbidatua bazara EditText bat eta gordetzeko botoi bat erakutsiko dizkizula eta, aldiz, datuen iraunkortasunean gordetako izen bat badago, agurtu egingo zaituela eta memoria-eremu hori ezabatzeko botoi bat izango duela.

```

fun showProfile(){
    LbIzena.visibility = View.VISIBLE
    //Log.d("Oharra", SharedApp.prefs.name)
    LbIzena.text = prefs.name
    Ezabatu.visibility = View.VISIBLE
    Izena.visibility = View.INVISIBLE
    Gorde.visibility = View.INVISIBLE
}

fun showGuest(){
    LbIzena.visibility = View.INVISIBLE
    Ezabatu.visibility = View.INVISIBLE
    Izena.visibility = View.VISIBLE
    Gorde.visibility = View.VISIBLE
}

```

Ikusgarritasunari buruz hitz egigo dugu Osagai bat hiru egoeratan egon daiteke ikusgai, ikusezin eta gone.

- ✓ **Ikusgai:** Osagaia pantailan ikusten da, besterik adierazi ezean aukera hau aktibatuta dator.
- ✓ **Ikusezina:** osagaia ez da ikusten, baina pantailan dago oraindik; beraz, harekin lan egiten jarrai daiteke, adibidez, osagai ikusezin baten eskuinaldean zerbait jartzea.
- ✓ **Gone:** osagaia EZ dago pantailan, eta, beraz, ez dago interakziorik.

Kontuan izan showProfile metodoak balio bat ematen diola LbIzena-ri, eta horrek esan nahi du shared preference-etan sartzen ari dela. Horretarako, SharedApp klaseari (object konpania duena), hitz egiten ari garen objektuari eta hasieran ikusi genuen bezala aldatu dugun name atributuari deitzen diogu, get bat egiten dugunean (biltegitratzen duen informazioa ateratu) gure shared preference SHARED_NAME balioa eska diezaiola.

Orain, name-n informazioa dagoen egiaztatzeko metodo bat behar dugu, erabiltzaileak bere izena gorde duen egiaztatzeko.

```

fun isSavedName():Boolean{
    val myName = prefs.name
    //Log.d("Oharra", myName)
    return myName != EMPTY_VALUE
}

fun configView(){
    if(isSavedName()) showProfile() else showGuest()
}

```

Amaitzeko, dagokion botoia sakatzen dugun bakoitzean soilik gorde edo ezabatu behar dugu erabiltzailearen informazioa, eta, jarraian, berriro deitu behar dugu configView-ra (), dagokion ikuspegia erakutsi dezan.

```
Gorde.setOnClickListener {  
    prefs.name = Izena.text.toString()  
    configView() }  
Ezabatu.setOnClickListener {  
    prefs.name = EMPTY_VALUE  
    configView()}  
}
```

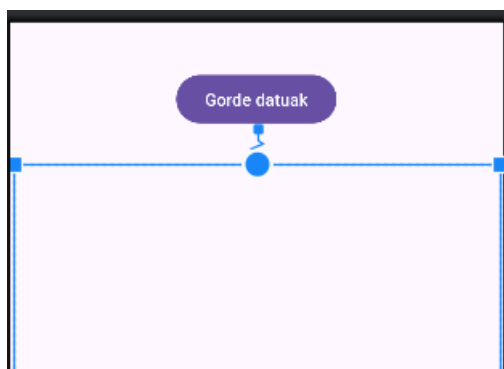
PRAKTIKA 07-01

Testu-fitxategien bidez

Atal honetan EditText kontrol batean oharrak gordetzeko aukera emango duen programa bat egingo da eta botoi bat sakatzen denean EditText datuak "notas.txt" izeneko testu-fitxategi batean biltegitratuko da.

Programan sartzen den bakoitzean, "notas.txt" testu-fitxategia dagoen egiaztatu, irakurri eta datuak EditText-en biltegitratuko dira.

Ondorengo itzura duen interfazea sortuko dugu:



Gorde Datuan azpian dagoen elementua multiline da.

Ikus dezagun lehendabizi nola grabatzen ditugun datuak testu-fitxategi batean. "Gorde" botoia sakatzean GordeFitxategian exekutatu da.

```
Gorde.setOnClickListener {  
    GordeFitxategian()  
}
```

OutputStreamWriter klaseko objektu bat sortuko dugu, eta klase horretako eraikitzaileari ActionBarActivity klaseari dagokion openFileOutput metodoa itzultzen duen datua bidaliko diogu. Parametro gisa testu-fitxategiaren izena eta irekitzeko modua pasatuko dizkiogu.

```
val fitxategia = OutputStreamWriter(openFileOutput( name: "notas.txt", MODE_PRIVATE))
```

Jarrian, fitxategia behar bezala sortu bada, write metodora deituko dugu eta grabatu beharreko string-a pasatuko diogu. Kasu honetan, EditText-etik aterako ditugu datuak:

```
archivo.write(Notak.getText().toString())
```

Write metodoarekin gorde ondoren, flush metodoari deituko diogu, bufferrean gera daitezkeen datu guztiak irauli ditzan, eta fitxategia itxiko dugu:

```
archivo.flush()  
archivo.close()
```

Hori guztia try/catch batean itxita dago, fitxategia irekitzean akatsen bat gertatzen den egiaztatzeko.

```
catch (e: IOException) {  
    val t = Toast.makeText(context: this, text: "Arazoren bat egon da Fitxategia irekitzean", Toast.LENGTH_SHORT)  
    t.show()  
}
```

Azkenik, aldi baterako mezu bat agertuko zaigu pantailan, Toast klasea erabiliz, abisu horrek datu guztiak ondo gorde direla adieraziko digu:

```
val t = Toast.makeText(context: this, text: "Datuak Gorde dira", Toast.LENGTH_SHORT)  
t.show()  
finish()
```

onCreate metodoaren helburua da testu-fitxategia badagoen egiaztatzea, hura irakurtzea eta edukia EditText-en erakustea.

Lehenik, Activity-k sortutako fitxategi guztien zerrenda lortuko dugu. Gure adibidean zero edo bat izan daiteke:

```
val fitxategiak = fileList()
```

String motako bektorean "notas.txt" fitxategia dagoen egiaztatzen duen metodo bati deituko diogu:

```
if (ExistitzenDa(fitxategiak, archbusca: "notas.txt"))
```

True itzultzen badu, InputStreamReader klaseko objektu bat sortuko dugu, eta klase horren eraikitzaileari openFileInput metodoaren bidez itzulitako datua pasatuko diogu:

```
val fitxategia = InputStreamReader(openFileInput( name: "notas.txt"))
```

BufferedReader klaseko objektu bat sortu eta InputStreamReader klaseko objektuaren erreferentzia emango diogu eraikitzaileari:

```
val br = BufferedReader(archivo)
```

Testu-fitxategiaren lehen lerroa irakurriko dugu:

```
var lerroa = br.readLine()
```

String huts bat abiaraziko dugu:

```
var textua = ""
```

BufferedReader klaseko readLine metodoak String bat itzultzen duen bitartean:

Stringari lotuko diogu lerro-jauzi baten ondoan

Hurrengo lerroa irakurriko dugu

```
while (lerroa != null) {  
    textua = textua + lerroa + "\n"  
    lerroa = br.readLine()  
}
```

BufferedReader eta InputStreamReader klaseko close metodoari deitzen diegu:

```
br.close()
```

```
fitxategia.close()
```

EditText testu-fitxategiko datu guztiak dituen String-aren edukiarekin kargatuko dugu:

```
Notak.setText(textua)
```

ExistitzenDa funtzioara String motako bektore bat iritsiko da. For baten barruan, bektoreko string bakoitzarekin bilatu beharreko string-a egiaztatuko dugu; aurkitzen badugu, true itzuliko du. For osoan zehar, for-etik kanpo aurkitu gabe, huts egiten dugu:

```
private fun ExistitzenDa(fitxategiak: Array<String>, bilatuFitx: String): Boolean {  
    for (f in fitxategiak.indices) if (bilatuFitx == fitxategiak[f]) return true  
    return false  
}
```

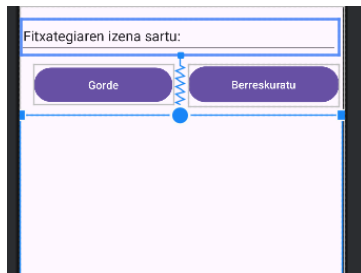
PRAKTIKA 07-02

SD Tarjeta

Aurreko kasuan ikusi dugu nola sortu eta irakurri testu-fitxategi bat Android ekipoaren barne-memorian. Egoera batzuetan erabilgarria izan liteke datuak SD txartel batean biltegiratzea (kontuan izan Android gailu guztiek ez dutela ezaugarri hori), SD txartela emanez fitxategiak beste pertsona batzuekin partekatzeko gaitasun edo erraztasun handiagoa dutelako.

Hau frogatzeko ondorengo aplikazioa garatuko dugu: Fitxategi baten izena eta edukia sartzea ahalbidetuko duen programa bat egitea. Botoi bat sakatzean sartutako datuak grabatzeko aukera ematea. Bigarren botoi bat jartzea, testu-fitxategiko datuak berreskuratzeke. Fitxategiak SD txartel batean grabatzea.

Ondorengo itzura duen interfazea sortuko dugu:



Botoien azpian dagoen elementua multiline da.

SD txartel batean kokatutako testu-fitxategi batean datuak grabatzeko metodoa, hasieran, txartelaren erraiz direktorioa `getExternalFilesDir` metodoaren bidez lortzen da, eta `File` klaseko objektu bat itzultzen da.

```
val file = File(getExternalFilesDir(type: null), FitxIzena)
val osw = OutputStreamWriter(
    FileOutputStream(file)
```

`File` klaseko objektu bat sortuko dugu, SD unitatearen bidea eta sortu beharreko fitxategiaren izena adierazita:

```
val file = File(getExternalFilesDir(type: null), FitxIzena)
```

Azkenik, barne-fitxategi baten atzipenaren antzera, `OutputStreamWriter` klaseko objektu bat sortzen dugu:

```
val osw = OutputStreamWriter(
    FileOutputStream(file)
```

`EditText`en edukia grabatuko dugu:

```
osw.write(edukia)
```

Artxiboa itxiko dugu:

```
osw.write(edukia)
osw.flush()
osw.close()
Toast.makeText(context: this, text: "Datu guztiak gorde dira", Toast.LENGTH_LONG).show()
SarreraD.setText("")
Textua.setText("")
} catch (ioe: IOException) {
    Toast.makeText(context: this, text: "Datuak ez dira gorde.", Toast.LENGTH_LONG).show()
}
```

Fitxategia irakurtzeko, berriz ere SD txartelaren erreferentzia lortuko dugu biltegiratze-unitateko patha lortzeko; algoritmoaren gainerakoa barne-fitxategi batekin ikusitakoaren antzekoa da:

```

fun DatuakBerreskuratu() {
    val file = File(getExternalFilesDir( type: null), SarreraD.getText().toString())
    try {
        val fIn = FileInputStream(file)
        val fitxategia = InputStreamReader(fIn)
        val br = BufferedReader(fitxategia)
        var lerroa = br.readLine()
        var testua = ""
        while (lerroa != null) {
            testua = "$testua$lerroa "
            lerroa = br.readLine()
        }
        br.close()
        fitxategia.close()
        Textua.setText(testua)
    } catch (e: IOException) {
        Toast.makeText( context: this, text: "Datuak ezin izan dira irakurri", Toast.LENGTH_LONG).show
    }
}

```

PRAKTIKA 07-03

DB

Orain arte, datuak modu iraunkorrean gordetzeko bi modu ikusi ditugu (testu-fitxategiak eta SharedPreferences klasea); orain, Androiden jatorrizko beste tresna bat ikusiko dugu, SQLite izeneko datu-base batean datuak gordetzeko.

SQLite Open Source datu-basea da, oso ezaguna gailu txiki askotan, Android kasuan ere.

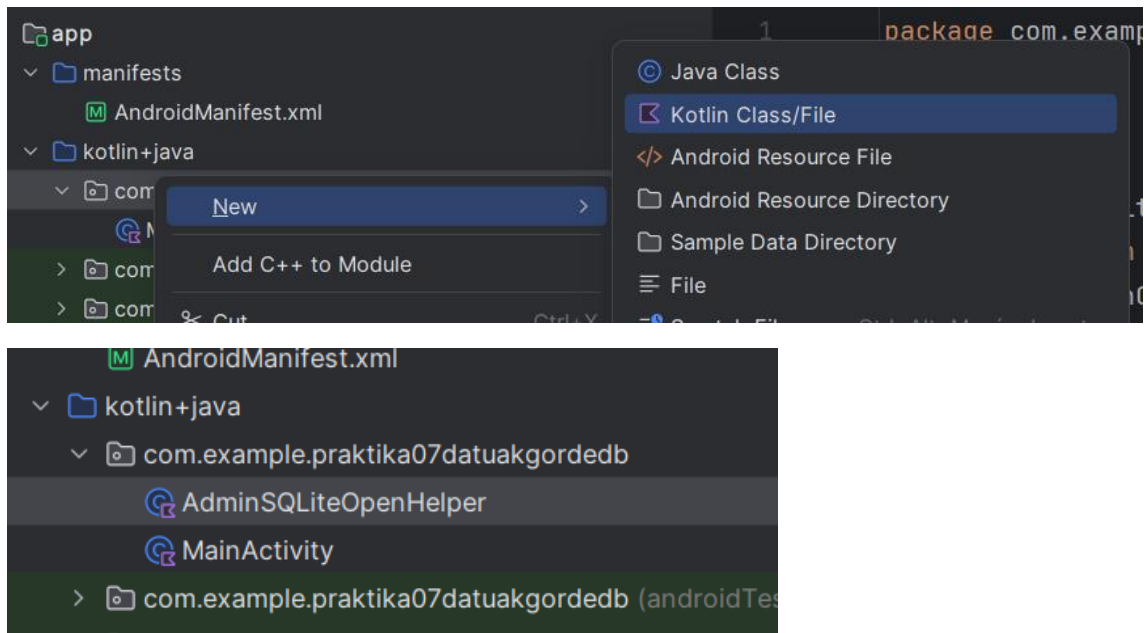
SQLite erabiltzeak dituen abantailak dira ez duela konfiguraziorik behar, ez duela datu-base-zerbitzaririk prozesu bereizi batean exekutatzen dela eta nahiko erraza dela hura erabiltzea.

Nola funtzionatzen duen frogatzeko ondorengo aplikazioa garatuko dugu: Artikuluen datuak biltegiratzeko aukera emango duen programa bat egitea. Artikuluen taula sortu eta kodea, artikuluen deskribapena eta prezioa eremuak definitu.

Programak honako hauek ahalbidetu behar ditu:

- 1 - Artikulu-karga.
- 2 - Kodearekin kontsultatu.
- 3.- Deskribapenaren bidez kontsultatu.
- 4 - Artikulu bat ezabatzea bere kodea sartuz.
- 5.- Deskribapena eta prezioa aldatzea.

Lehenik eta behin, SQLiteOpenHelper-etik heredatzen den klase bat sortuko dugu. Klase horrek aukera ematen digu datu-basea sortzeko eta hasierako taulen eta datuen egitura eguneratzeko. Android Studiotik klase berri bat sortzeko, New -> Kotlin Class hautatuko dugu eta AdminSQLiteOpenHelper izena emango diogu



Sortuko dugun datu-basea administratzea helburu duen klase hori kodetu behar dugu orain. Lehenik, gure klasea SQLiteOpenHelper klasetik heredatuko dugu:

```
import android.database.sqlite.SQLiteOpenHelper

class AdminSQLiteOpenHelper: SQLiteOpenHelper {
}
```

SQLiteOpenHelper klaseak bi metodo inplementatzea eskatzen du, nahitaez, onCreate eta onUpgrade; Android Studioak kodetzean lagun diezagula egin dezakegu, eta, horrela, ez ditugu guk tindatu behar: "ALT" eta "ENTER" teklak sakatzen ditugu, kurtsorea "AdminSQLiteOpenHelper" klasearen izenaren gainean edukita, eta menu bat agertzen zaigu, "Implement Methods" hautatzeko, eta, ondoren, elkarrizketa bat, inplementatu beharreko metodoak hautatzeko:

```
2
3 import android.database.sqlite.SQLiteOpenHelper
4
5 class AdminSQLiteOpenHelper:SQLiteOpenHelper {
6 }
```

Implement members
Make 'AdminSQLiteOpenHelper' 'abstract'
Safe delete 'AdminSQLiteOpenHelper'

Implement Members

android.database.sqlite.SQLiteOpenHelper

- onCreate(p0: SQLiteDatabase!): Unit
- onUpgrade(p0: SQLiteDatabase!, p1: Int, p2: Int): Unit

```
1 package com.example.praktika07datuakgordedb
2
3 import android.database.sqlite.SQLiteDatabase
4 import android.database.sqlite.SQLiteOpenHelper
5
6 class AdminSQLiteOpenHelper:SQLiteOpenHelper {
7     override fun onCreate(p0: SQLiteDatabase?) {
8         TODO( reason: "Not yet implemented")
9     }
10
11     override fun onUpgrade(p0: SQLiteDatabase?, p1: Int, p2: Int) {
12         TODO( reason: "Not yet implemented")
13     }
14 }
```

Bi metodoko klase bat izango dugu.

Baina oraindik ere errore bat markatzen digu; izan ere, SQLiteOpenHelper klase aitzak parametroak dituzten eraikitzaileak inplementatzen ditu. Orduan, klase horretako eraikitzailea definitu behar dugu, eta, horretarako, kurtsorea "AdminSQLiteOpenHelper" klasearen izenaren gainean egonda, berriro ere "ALT" eta "Enter" teklak sakatuko ditugu, eta agertzen den kontaktu-menuan "Create constructor matching super" hautatuko dugu. Guraso-klasea inplementatzen duen elkarrizketa bat agertzen da eraikitzaile guztiekin:

```
import android.content.Context
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

class AdminSQLiteOpenHelper(
    context: Context?,
    name: String?,
    factory: SQLiteDatabase.CursorFactory?,
    version: Int
): SQLiteOpenHelper(context, name, factory, version) {
    override fun onCreate(p0: SQLiteDatabase?) {
        TODO(reason: "Not yet implemented")
    }

    override fun onUpgrade(p0: SQLiteDatabase?, p1: Int, p2: Int) {
        TODO(reason: "Not yet implemented")
    }
}
```

onCreate metodoren barruan gure taula sortuko dugu:

```
override fun onCreate(db: SQLiteDatabase) {
    db.execSQL(sql: "create table articulos(codigo int primary key,descripcion text,precio real)");
}
```

(Taularen egitura aldatu nahi badugu, onUpgrade metodoan egin behar dugu).

Orain gure aplikazioa diseinatzera pasako gara, eta interfazeak ondorengo itzura izan behar du:

Kodigoa

Deskribapena

Prezioa

Gorde

Konsultatu Kodea

Konsultatu Deskribapena

Ezabatu

Aldatu

DATUAK GORDE

"Gorde" botoia sakatzen denean, "Create" metodoa exekutatzen da. Metodo honetan, lehenik eta behin, aurrez planteatutako klaseko objektu bat sortzen dugu, eta this eraikitzaileari (egungo Activity-ren erreferentzia), "administracion" (halakorik ez badago, sortuko dugun datu-basearen izena da), ondoren, null eta bat balio bat pasatzen diogu, datu-basearen lehen bertsioa dela adierazteko

```
val admin = AdminSQLiteOpenHelper(  
    context: this,  
    name: "administracion", factory: null, version: 1  
)  
val bd = admin.writableDatabase
```

AdminSQLiteOpenHelper klaseko objektu bat sortu ondoren, SQLiteDatabase klaseko objektu bat sortuko dugu getWritableDatabase metodora deituta (datu-basea irakurtzeko eta idazteko moduan irekitzen da).

ContentValues klaseko objektu bat sortu eta put metodoaren bidez kargatu beharreko eremu guztiak hasieratuko ditugu.

```

val kod: String = kodigoa.getText().toString()
val deskri: String = Deskrib.getText().toString()
val pre: String = Prezioa.getText().toString()
val registro = ContentValues()
registro.put("codigo", kod)
registro.put("descripcion", deskri)
registro.put("precio", pre)

```

Ondoren, SQLiteDatabase klaseko insert metodoari deituko diogu, eta lehenengo parametroan taularen izena sartuko dugu, bigarren parametro gisa null bat, eta, azkenik, jada abiarazita dagoen ContentValues klasearen objektua (metodo horrek eragiten du artikuluen taulan beste errenkada bat sartzea, administrazio izeneko datu-basean).

```

bd.insert( table: "articulos", nullColumnHack: null, registro)

```

Jarraian, EditText ezabatu eta mezu bat erakutsiko dugu, erabiltzaileak jakin dezan datuak behar :

```

bd.close()
kodigoa.setText("")
Deskrib.setText("")
Prezioa.setText("")
Toast.makeText( context: this, text: "Artikuluaren datuak kargatu dira", Toast.LENGTH_LONG).show()

```

KONTSULTA BURUTU KODEAREN ARABERA

Kontsultatu kodea botoia sakatzean ReadKodigoa metodoa exekutatzen da. Kontsulta-metodoan, lehenik eta behin, AdminSQLiteOpenHelper klaseko objektu bat sortzen dugu, eta datu-basetik erreferentzia bat lortzen dugu getWritableDatabase metodora deituta.

```

val admin = AdminSQLiteOpenHelper(
    context: this,
    name: "administracion", factory: null, version: 1
)
val bd = admin.writableDatabase

```

Jarraian, KurtsoRe klaseko aldagai bat definitu eta rawQuery izeneko metodoaren bidez itzulitako balioarekin hasieratuko dugu.

```

val lerroa = bd.rawQuery(
    sql: "select descripcion,precio from articulos where codigo=$Kod", selectionArgs: null
)

```

Kurtsore klaseak, kasu honetan, errenkada bat edo zero errenkada gordetzen ditu (bat artikuluen taulan dagoen kode bat sartu badugu), Kurtsore klaseko `moveToFirst()` metodoari deitzen diogu, eta `true` itzultzen da kodea sartuta duen artikulua bat badago; bestela, `zero` itzultzen da.

```
if (lerroa.moveToFirst()) {
```

Kontsultatu nahi ditugun datuak berreskuratzeko `getString` metodoari deituko diogu eta berreskuratu beharreko eremuaren posizioa pasatuko diogu (zeroan zenbatzen hasten da; adibide honetan, zero zutabeak deskribapena eremua adierazten du, eta 1 zutabeak prezioa eremua adierazten du).

```
Deskrib.setText(lerroa.getString(0))
```

```
Prezioa.setText(lerroa.getString(1))
```

Ez badago kode hori duen lerrorik orduan erabiltzaileari abisatu behar zaio.

```
else Toast.makeText( context: this, text: "Ez dago kode hori duen artikulurik", Toast.LENGTH_LONG).show()
```

Behin hau bukatuta datu basea itxi beharko da.

```
bd.close()
```

KONTSULTA BURUTU DESKRIBAPENAREN ARABERA

Kontsultatu deskribapena botoia sakatzean `ReadDeskribapena` metodoa exekutatzen da. Metodo hau aurrekoaren antzeko da. Ikasleen eskutean geratzen da.

DATUAK EZABATU

Erregistro bat edo gehiago ezabatzeko, `SQLiteDatabase` klaseak metodo bat du: lehenengo parametroan taularen izena emango diogu, eta bigarrenean, taulako errenkada ezabatzeko bete behar den baldintza. `Delete` metodoa zenbaki oso bat itzultzen da, ezabatutako erregistro kopurua adierazten duena:

```
val cant = bd.delete( table: "articulos", whereClause: "codigo=$cod", whereArgs: null)
bd.close()
```

```
if (cant == 1) Toast.makeText( context: this, text: "Artikuluak ezabatu da", Toast.LENGTH_LONG).show()
else Toast.makeText( context: this, text: "Ez dago kode hori duen artikulurik", Toast.LENGTH_LONG).show()
```

DATUAK ALDATU

Datu-aldaketan, `ContentValues` klaseko objektu bat sortu behar dugu, eta, put metodoaren bidez, aldatu egingo den eremu bakoitzerako balioak biltegitatu.

```

val kod: String = kodigoa.getText().toString()
val deskri: String = Deskrib.getText().toString()
val pre: String = Prezioa.getText().toString()
val registro = ContentValues()
registro.put("codigo", kod)
registro.put("descripcion", deskri)
registro.put("precio", pre)

```

Ondoren, SQLiteDatabase klaseko update metodoari deitzen zaio, taularen izena, ContentValues klasearen objektua eta where baldintza (adibide honetan parametro guztia ez da erabiltzen) pasatuz.

```

val kopurua = bd.update(table: "articulos", registro, whereClause: "codigo=$kod", whereArgs: null)
bd.close()
if (kopurua == 1) Toast.makeText(context: this, text: "Datuk aldatu dira.", Toast.LENGTH_SHORT).show()
else Toast.makeText(context: this, text: "Ez dago kode hori duen artikulurik", Toast.LENGTH_LONG).show()

```